

CocinaP

Sistema de Seguridad Inteligente para Cocina

Reporte Tecnico y Creativo

Version 1.0.1

Julio 2026

1. Resumen Ejecutivo

CocinaP es un sistema de seguridad para cocina que utiliza inteligencia artificial para detectar y alertar sobre situaciones de riesgo en tiempo real. Emplea el modelo YOLO11n (You Only Look Once) para la detección de fuego, humo, personas y objetos en la estufa, analizando el riesgo de forma continua.

El sistema consta de dos componentes principales: una aplicación nativa para Windows construida con Python y PySide6 que funciona como servidor local, y una aplicación móvil companion para Android desarrollada en Flutter que permite el monitoreo remoto vía WiFi. Ambas aplicaciones se comunican a través de una API REST, un stream de video MJPEG en tiempo real, y eventos Server-Sent (SSE).

El objetivo principal es reducir el riesgo de incendios en la cocina mediante la detección temprana de fuego, humo, y situaciones de cocina desatendida, emitiendo alertas sonoras locales y notificaciones push opcionales vía Firebase Cloud Messaging (FCM).

2. Que Hace el Programa

CocinaP funciona como un vigilante automatico de la cocina. Utiliza la camara web del computador para capturar video en tiempo real y lo procesa mediante un modelo de vision artificial (YOLO11n) que identifica los siguientes elementos:

2.1 Detecciones Principales

- Fuego: Detecta llamas y regiones de fuego en el area de la cocina, calculando la cobertura de fuego sobre la superficie visible.
- Humo: Identifica presencia de humo, midiendo su concentracion y cobertura en la escena.
- Personas: Reconoce la presencia de personas en la cocina para determinar si hay alguien supervisando.
- Vasijas en la estufa: Detecta ollas, sartenes y otros objetos sobre la zona de la estufa, para identificar cocina desatendida.
- Zona de estufa: El usuario puede configurar un area especifica en la imagen que corresponde a la estufa, permitiendo detecciones contextuales.

2.2 Analisis de Riesgo

El analizador de riesgo procesa las detecciones del modelo YOLO y las clasifica en cuatro niveles de severidad:

- CRITICO: Fuego activo o humo espeso con alta cobertura.
- ALTO: Humo significativo o fuego incipiente cerca de la estufa.
- MEDIO: Cocina desatendida por tiempo prolongado o humo leve.
- BAJO: Presencia de humo muy tenue o detecciones aisladas.

2.3 Acciones del Sistema

- Alarma sonora local: El sistema emite pitidos audibles en la PC mediante la funcion Beep de Windows, con patrones distintivos para fuego, humo y cocina desatendida.
- Notificaciones push (FCM): Si esta configurado, envia notificaciones push a dispositivos Android registrados via Firebase Cloud Messaging.
- Actualizaciones en tiempo real: El servidor web integrado transmite el estado del sistema via SSE (Server-Sent Events) y el video via MJPEG stream.

2.4 Interfaz Grafica Nativa (Windows)

La aplicacion nativa de Windows presenta tres pestanas principales:

- Camara: Muestra el video en vivo con las detecciones superpuestas, el estado actual del riesgo y los FPS del sistema.
- Configuracion: Permite ajustar los parametros del detector (confianza, IoU, area de la estufa) y del analizador (tiempos, umbrales, FCM).
- Alarmas: Historial de eventos y alertas generadas por el sistema.

2.5 Aplicacion Movil (Android)

La aplicacion companion para Android permite:

- Descubrimiento automatico del servidor via mDNS/Zeroconf en la red local.

- Visualización del stream de video en vivo.
- Tarjetas de estado con conteo de fuego, humo, personas y vasijas.
- Configuración remota de parámetros del sistema.
- Historial de alarmas con detalles de severidad y tipo.
- Servicio en foreground que mantiene la app activa para recibir notificaciones de alarma incluso en background.

3. Necesidades Tecnicas

3.1 Hardware Requerido

Para el funcionamiento del sistema se requiere:

- Computador con Windows 10/11, procesador de 64 bits, minimo 8 GB de RAM (recomendado 16 GB para rendimiento optimo).
- Camara web (integrada o USB) con resolucion minima de 640x480, recomendada 1280x720.
- Almacenamiento: 1.5 GB de espacio disponible para la instalacion completa.
- Conexion WiFi local (red 2.4 GHz o 5 GHz) para comunicacion con el dispositivo movil.
- Dispositivo Android 8.0+ (API 26+) para la aplicacion companion.

3.2 Software y Dependencias (Backend - Windows)

El servidor backend esta desarrollado en Python 3.14.6 y requiere:

- Python 3.14.6 (CPU-only, sin CUDA)
- OpenCV (opencv-python-headless) - Procesamiento de imagenes y video
- PySide6 - Interfaz grafica nativa de Windows
- Ultralytics (YOLO11n) - Modelo de deteccin de objetos
- Pillow (PIL) - Procesamiento de imagenes adicional
- matplotlib - Visualizaciones internas
- sympy - Dependencia matematica de ultralytics
- zeroconf - Descubrimiento mDNS en red local
- firebase-admin (opcional) - Notificaciones push FCM
- numpy - Computo numerico
- pyinstaller (build) - Empaquetado en ejecutable unico

3.3 Software y Dependencias (Frontend - Movil)

La aplicacion movil esta desarrollada en Flutter 3.x con Dart 3.x:

- Flutter SDK 3.x con canal stable
- Dart SDK 3.x
- Librerias principales: http, provider, shared_preferences, multicast_dns
- Firebase Core y Firebase Messaging (notificaciones push)
- flutter_local_notifications (notificaciones locales Android)
- flutter_background_service (servicio en foreground para Android)
- Android SDK 36, NDK 28, build-tools 36.0.0, JDK 21
- Gradle 8.x para el build system Android

3.4 Red y Conectividad

- Protocolos: HTTP (API REST), MJPEG (stream de video), SSE (eventos en tiempo real), mDNS (descubrimiento automatico)

- Puerto predeterminado: 8080 (configurable)
- Descubrimiento: Zeroconf con servicio anunciado como `_cocinap._tcp.local`.
- Firewall: Requiere regla de entrada para el puerto 8080 en Windows Firewall

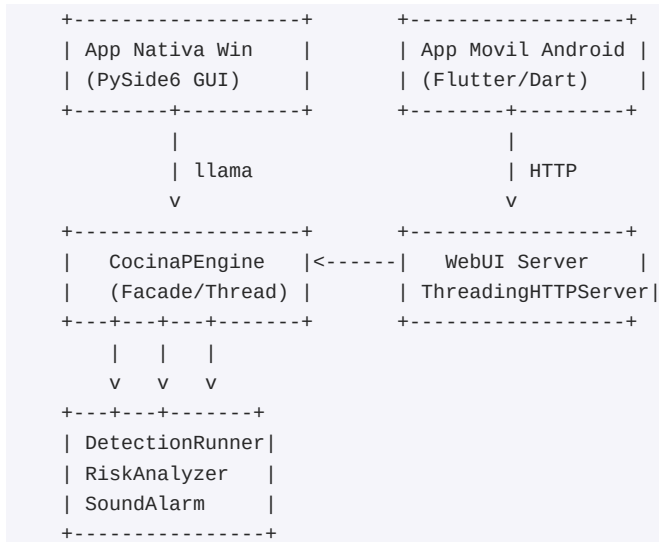
3.5 Requisitos de Instalacion y Distribucion

- Windows: Instalador Inno Setup (~92 MB) que despliega el ejecutable compilado con PyInstaller, incluyendo el modelo YOLO y todas las dependencias.
- Android: APK de la aplicacion companion (~50-150 MB segun tipo de build).
- Firebase: Archivo google-services.json (opcional, solo si se desea FCM).

4. Arquitectura del Sistema

4.1 Diagrama de Componentes

El sistema sigue una arquitectura modular con un nucleo compartido (motor) que sirve tanto a la interfaz grafica nativa como al servidor web embebido:



4.2 Flujo de Datos

1. La camara captura frames a resolucion completa (ej. 1280x720).
2. DetectionRunner recibe el frame via submit_frame() o callback.
3. El detector YOLO11n procesa el frame a resolucion reducida (320x320) y devuelve las detecciones.
4. Las detecciones pasan al RiskAnalyzer que evalua la severidad y determina si debe activar la alarma.
5. SoundAlarm emite pitidos si hay alerta activa.
6. El frame con las detecciones dibujadas se muestra en la GUI nativa.
7. WebUI sirve el frame (sin overlays, downscaled a 480px) via MJPEG para la app movil.
8. WebUI publica el estado via SSE y almacena el historial de alarmas.
9. WebUI envia notificaciones FCM si hay alerta critica/alta.

4.3 Modulos del Sistema

- cocinap/config.py: Gestion de configuracion persistente en JSON.
- cocinap/engine.py: Fachada del sistema, orquesta todos los modulos.
- cocinap/detector/: Detector YOLO y DetectionRunner (hilo background).
- cocinap/analyzer/: RiskAnalyzer que evalua contexto y severidad.
- cocinap/alarm/: SoundAlarm con pitidos asincronos vía kernel32.Beep.
- cocinap/utils/: Funciones de dibujo (draw_detections) y visualizacion.
- cocinap/webui.py: Servidor HTTP embebido (API, MJPEG, SSE, FCM, mDNS).
- cocinap/app.py: Aplicacion PySide6 con 3 pestanas (Camara, Config, Alarmas).

4.4 API REST (Endpoints)

GET	/	- Información básica del servidor
GET	/api/stream	- Stream MJPEG de video en vivo
GET	/api/events	- Server-Sent Events (alarmas en tiempo real)
GET	/api/status	- Estado actual (detecciones, alertas)
GET	/api/config	- Configuración actual del sistema
GET	/api/alarms	- Historial de alarmas (últimas 50)
GET	/api/info	- Información del servidor (version, uptime, IPs)
POST	/api/config	- Actualizar configuración
POST	/api/register_token	- Registrar token FCM
POST	/api/unregister_token	- Desregistrar token FCM

5. Necesidades Creativas

5.1 Diseño de la Interfaz Nativa (Windows)

La interfaz gráfica nativa está construida con PySide6 y debe cumplir con los siguientes principios de diseño:

- Claridad visual: La pestaña de cámara debe mostrar el video en vivo de forma predominante, con superposiciones mínimas pero informativas (bounding boxes de color según el tipo de detección: rojo para fuego, naranja para humo, verde para personas).
- Jerarquía de información: El estado de riesgo debe ser visible de inmediato, con un indicador de color (verde = seguro, amarillo = atención, naranja = precaución, rojo = peligro) y texto descriptivo.
- Feedback inmediato: Cualquier cambio en la configuración debe reflejarse instantáneamente, con indicadores visuales de que el cambio fue aplicado.
- Consistencia: Uso de iconos estándar de Windows, tipografía del sistema, y patrones de interacción familiares para el usuario de Windows.
- Modo oscuro compatible: La paleta de colores debe funcionar correctamente en configuraciones de tema oscuro de Windows.

5.2 Diseño de la Aplicación Móvil (Android)

La aplicación companion para Android sigue Material Design 3 con los siguientes principios:

- Primera pantalla - Discovery: Una interfaz limpia que muestra los servidores disponibles en la red local con un indicador visual de conexión. El usuario debe poder identificar rápidamente su servidor CocinaP.
- Dashboard - Cámara en vivo: El video stream debe ocupar la mayor parte de la pantalla. Las tarjetas de estado (StatusCard) debajo del video muestran los contadores de fuego, humo, personas y vajillas con iconos intuitivos y colores semafóricos.
- Pantalla de Alarmas: Lista cronológica inversa con código de colores por severidad. Cada alarma muestra tipo, severidad, mensaje y timestamp.
- Pantalla de Configuración: Formulario con sliders y campos numéricos organizados por sección, con botón de guardar que envía los cambios al servidor.
- Pantalla de Ajustes (Settings): Opciones locales del dispositivo móvil como intervalo de refresco, notificaciones, y preferencias de conexión.

5.3 Experiencia de Usuario (UX)

- Tiempo de respuesta: El sistema debe mostrar el video en la app móvil con una latencia máxima de 2-3 segundos desde la captura de cámara.
- Notificaciones: Las alertas críticas deben generar notificaciones con vibración y sonido distintivo en el dispositivo móvil.
- Reconexión automática: Si la conexión WiFi se pierde, la app móvil debe intentar reconectarse automáticamente al servidor.
- Configuración intuitiva: Los parámetros técnicos (confianza YOLO, IoU) deben presentarse con descripciones claras en español y rangos predefinidos que eviten valores inválidos.
- Feedback de estado: La barra superior debe mostrar el estado de conexión con el servidor (conectado/desconectado/buscando).

5.4 Identidad Visual

La identidad visual de CocinaP se basa en los siguientes elementos:

- Nombre: CocinaP (Cocina + Prevention/Security).
- Paleta de colores: Azul corporativo (#193C78) como color principal, con acentos rojo (#DC3545) para peligro, naranja (#FD7E14) para advertencia, verde (#28A745) para seguro.
- Iconografía: Iconos de cocina y seguridad (llama, humo, persona, olla, escudo) utilizando la librería Material Icons.
- Tono de comunicación: Profesional y directo, con términos técnicos mínimos explicados para el usuario no experto.

6. Seguridad y Privacidad

CocinaP opera exclusivamente en la red local, sin enviar datos a servidores externos. Las consideraciones de seguridad incluyen:

- Todo el procesamiento de video se realiza localmente en el computador del usuario. No se transmiten imágenes a la nube ni a terceros.
- La comunicación entre la app móvil y el servidor se realiza vía HTTP en la red local (WiFi). No hay exposición a Internet.
- El servidor HTTP embebido solo escucha en la interfaz de red local (0.0.0.0:8080), accesible únicamente desde dispositivos en la misma subred.
- El descubrimiento mDNS solo funciona dentro de la red local.
- Firebase Cloud Messaging es opcional y solo se utiliza para notificaciones push. No se envían datos de video ni información personal a Firebase.
- No se requiere registro de usuario, cuenta, ni autenticación para operar.

7. Requisitos para Distribucion

7.1 Componente Windows

- Sistema operativo: Windows 10/11 de 64 bits.
- No requiere Python instalado (ejecutable autonomo compilado con PyInstaller).
- Instalador: CocinaP_Setup_v1.0.1.exe (~92 MB) generado con Inno Setup.
- El instalador despliega el ejecutable, el modelo YOLO, y todas las DLLs necesarias.
- Espacio en disco: ~850 MB para la instalacion completa (dist).
- Permisos: Requiere permisos de administrador para la instalacion.

7.2 Componente Movil

- Dispositivo Android 8.0+ (API 26+).
- APK firmado con debug o release keys.
- Permisos requeridos: INTERNET, FOREGROUND_SERVICE, FOREGROUND_SERVICE_DATA_SYNC, POST_NOTIFICATIONS, WAKE_LOCK.
- Requiere habilitar 'Instalar apps de orígenes desconocidos' para instalar el APK fuera de Google Play.
- Opcional: google-services.json de Firebase Console para FCM.

7.3 Red

- Ambos dispositivos (PC y movil) deben estar en la misma red WiFi.
- Puerto 8080 debe estar abierto en el firewall de Windows. Si no se puede agregar la regla, conexion manual via IP.
- Para conexion manual, el movil necesita la IP local del PC.

8. Conclusion

CocinaP representa una solución accesible y efectiva para la prevención de incendios en el hogar, combinando tecnologías modernas de visión artificial con un diseño centrado en el usuario. Su arquitectura modular permite mantenimiento y evolución independiente de cada componente.

Al operar completamente en la red local y no depender de servicios en la nube, CocinaP garantiza la privacidad del usuario y ofrece un funcionamiento confiable incluso sin conexión a Internet. Las notificaciones push vía FCM son opcionales y representan el único componente que requiere conectividad externa.

Las principales áreas de mejora identificadas incluyen la optimización del stream de video para dispositivos móviles (reducción de resolución implementada), la mejora del parser MJPEG en la app Android (corregido), y la resolución de problemas de descubrimiento mDNS en redes con firewall restrictivo.

CocinaP está listo para distribución en fase beta, permitiendo que usuarios reales validen el sistema en entornos domésticos y proporcionen retroalimentación para futuras mejoras.